UNIVERSIDAD NACIONAL SAN AGUSTÍN DE AREQUIPA
FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS
ESCUELA PROFESIONAL DE CIENCIA DE LA COMPUTACIÓN



Laboratorio 5: Analizador Sintáctico Predictivo LL(1) SQL

Docente : Quispe Cruz, Marcela

Curso : Estructuras de Datos Avanzadas

Alumnos : Ccaso Idme, Moises Robert

Quispe Torrez, Job Lorenzo

Torres Chávez, Jhair Alejandro

Arequipa - Perú

2026

REPOSITORIO:
https://github.com/TorresJhair/SQL-Engine-EDA-2026B-/tree/main/src/parser_ll1

1. Introducción

En la fase anterior (Práctica 4), se diseñó e implementó el **analizador léxico (sql::Lexer)**, encargado de reconocer la secuencia ordenada de componentes léxicos (sql::Token) y clasificar lexemas inválidos sin colapsar. La presente Práctica 5 se enfoca en el diseño formal e implementación en C++17 de un **analizador sintáctico predictivo no recursivo LL(1) (sql::Parser)**.

Este componente **interactúa de forma no destructiva con el módulo léxico, consumiendo directamente su salida (sql::LexResult)**. Su propósito es determinar de forma unívoca si la sentencia cumple con la **Gramática Libre de Contexto (CFG)** formalizada, simulando el proceso de derivación sintáctica mediante una pila explícita y la **matriz de análisis M[A,a]**, generando diagnósticos con **ubicación precisa ante cualquier error sintáctico o léxico.**

2. Arquitectura

El proyecto está organizado de manera modular en C++17, estructurado para soportar el desarrollo progresivo de un motor de base de datos relacional (almacenamiento, indexación y front-end de procesamiento de consultas SQL):

```
/ (Raíz del proyecto)
├── CMakeLists.txt              # Configuración global del sistema de construcción CMake y suites de CTest
├── README.md                   # Documentación general del repositorio, guías de compilación y ejecución
├── src/                        # Código fuente del motor de base de datos y front-end SQL
│   ├── storage/                # Fase 1: Motor de almacenamiento físico y páginas ranuradas (Slotted Page)
│   ├── index/                  # Fase 1: Implementación del índice B+ Tree sobre disco
│   ├── lexer/                  # Fase 2.1: Analizador léxico SQL y modelo de tokens (Práctica 4)
│   ├── parser_ll1/             # Fase 2.2: Analizador sintáctico predictivo LL(1) no recursivo (Práctica 5)
│   ├── common/                 # Definiciones transversales, tipos y utilidades del sistema
│   ├── bench/                  # Benchmarks y métricas de rendimiento
│   └── main.cpp                # Punto de entrada principal interactivo de la base de datos
└── tests/                      # Batería de pruebas unitarias y de integración automatizadas
```

3. Modelo de tokens (Lexer)

El analizador sintáctico consume de forma secuencial y no destructiva los tokens emitidos por **sql::Lexer (Práctica 4)**. A continuación se formaliza el modelo de tokens completo implementado en src/lexer/token.hpp:

Categoría	Tipos (TokenType)	Lexemas reconocidos
Palabras reservadas	**KEYWORD_SELECT, KEYWORD_FROM, KEYWORD_WHERE, KEYWORD_AND, KEYWORD_OR**	**SELECT, FROM, WHERE, AND, OR (sin distinguir mayúsculas ni minúsculas)**
Identificadores y literales	**IDENTIFIER**	**Nombres de tablas y columnas formados por letras ASCII, dígitos y guion bajo ([a-zA-Z_][a-zA-Z0-9_]*), rechazando palabras con caracteres no ASCII**
Identificadores y literales	**INTEGER_LITERAL**	**Secuencias de dígitos decimales (ej. 42, 007)**
Identificadores y literales	**DECIMAL_LITERAL**	**Números con punto fraccionario y/o notación científica con signo de exponente (ej. 3.14, 1e5, 1e-10, 2.5E+3)**
Identificadores y literales	**STRING_LITERAL**	**Cadenas entre comillas simples con soporte de comilla escapada mediante ‘’ (ej. ‘hola mundo’, ‘it’’s’)**
Operadores relacionales	**EQUAL, NOT_EQUAL, LESS, GREATER, LESS_EQUAL, GREATER_EQUAL**	**=, <>, !=, <, >, <=, >=**
Operadores aritméticos	**PLUS, MINUS, SLASH, STAR**	**+, -, /, * (como operador aritmético de multiplicación dentro de expresiones)**
Delimitadores y puntuación	**COMMA, SEMICOLON, STAR, LPAREN, RPAREN**	**, ; * (como comodín de proyección total en SELECT), (,)**
Fin de entrada	**END_OF_FILE**	**Fin de entrada / marcador centinela $**

4. Formalización de la Gramática

4.1. Eliminación de la recursividad izquierda y factorización

- **Secuencias repetitivas:** Las listas y operadores asociativos por la izquierda se transformaron en recursiones derechas mediante símbolos de cola que derivan en ε (cols_tail, disy_tail, conj_tail, comp_tail, suma_tail, prod_tail).
- **Factorización de la proyección:** Se factorizó la proyección aislando STAR (*) frente a lista_cols.
- **Determinismo del paréntesis ()**: En SQL clásico, el token (introduce ambigüedad al poder iniciar una condición lógica agrupada (a = 1 OR b = 2) o una expresión aritmética (a + 1) > 2. Para mantener determinismo LL(1) puro sin requerir retroceso (backtracking), la gramática unifica la jerarquía haciendo que primario → LPAREN condicion RPAREN descienda a través de la cascada completa de precedencias:

Disy (OR) → Conj(AND) → Comp → Suma → Prod → Unario → Primario

Esto permite que (a + 1) descienda de forma natural hasta suma, resolviendo la ambigüedad y manteniendo la condición LL(1).

4.2. Gramática formal en notación BNF (39 producciones)

Símbolo inicial: ***consulta***. El símbolo **$** representa **END_OF_FILE** y **ε** la producción vacía.

```
None
    ( 1) consulta    → KEYWORD_SELECT proyeccion KEYWORD_FROM
    IDENTIFIER where_opt SEMICOLON
    ( 2) proyeccion  → STAR
    ( 3) proyeccion  → lista_cols
    ( 4) lista_cols  → IDENTIFIER cols_tail
    ( 5) cols_tail   → COMMA IDENTIFIER cols_tail
    ( 6) cols_tail   → ε
    ( 7) where_opt   → KEYWORD_WHERE condicion
    ( 8) where_opt   → ε
    ( 9) condicion   → disy
    (10) disy        → conj disy_tail
    (11) disy_tail   → KEYWORD_OR conj disy_tail
    (12) disy_tail   → ε
    (13) conj        → comp conj_tail
```

```
(14) conj_tail   → KEYWORD_AND comp conj_tail
(15) conj_tail   → ε
(16) comp        → suma comp_tail
(17) comp_tail   → op_comp suma
(18) comp_tail   → ε
(19) op_comp     → EQUAL
(20) op_comp     → NOT_EQUAL
(21) op_comp     → LESS
(22) op_comp     → GREATER
(23) op_comp     → LESS_EQUAL
(24) op_comp     → GREATER_EQUAL
(25) suma        → prod suma_tail
(26) suma_tail   → PLUS prod suma_tail
(27) suma_tail   → MINUS prod suma_tail
(28) suma_tail   → ε
(29) prod        → unario prod_tail
(30) prod_tail   → STAR unario prod_tail
(31) prod_tail   → SLASH unario prod_tail
(32) prod_tail   → ε
(33) unario      → MINUS unario
(34) unario      → primario
(35) primario    → IDENTIFIER
(36) primario    → INTEGER_LITERAL
(37) primario    → DECIMAL_LITERAL
(38) primario    → STRING_LITERAL
(39) primario    → LPAREN condicion RPAREN
```

4.3. Codificación de la Gramática Formal (*parser_ll1/grammar.cpp*)

Utilizando las abreviaturas **NT para No Terminales** y **T para Terminales**

```
( 1) consulta    → KEYWORD_SELECT proyeccion KEYWORD_FROM IDENTIFIER where_opt SEMICOLON
```

Se traduce a:

```
{NT::Consulta, {t(T::KEYWORD_SELECT), n(NT::Proyeccion), t(T::KEYWORD_FROM),
    t(T::IDENTIFIER), n(NT::WhereOpt), t(T::SEMICOLON)}},
```

```cpp
Grammar::Grammar() {
    // Gramatica LL(1) estricta (sin recursion izquierda y factorizada). La
    // numeracion de los comentarios coincide la producción
    productions_ = {
        // 1
        {NT::Consulta, {t(T::KEYWORD_SELECT), n(NT::Proyeccion), t(T::KEYWORD_FROM),
                        t(T::IDENTIFIER), n(NT::WhereOpt), t(T::SEMICOLON)}},
        // 2, 3
        {NT::Proyeccion, {t(T::STAR)}},
        {NT::Proyeccion, {n(NT::ListaCols)}},
        // 4
        {NT::ListaCols, {t(T::IDENTIFIER), n(NT::ColsTail)}},
        // 5, 6
        {NT::ColsTail, {t(T::COMMA), t(T::IDENTIFIER), n(NT::ColsTail)}},
        {NT::ColsTail, {}},
        // 7, 8
        {NT::WhereOpt, {t(T::KEYWORD_WHERE), n(NT::Condicion)}},
        {NT::WhereOpt, {}},
        // 9
        {NT::Condicion, {n(NT::Disy)}},
        // 10
        {NT::Disy, {n(NT::Conj), n(NT::DisyTail)}},
        // 11, 12
        {NT::DisyTail, {t(T::KEYWORD_OR), n(NT::Conj), n(NT::DisyTail)}},
        {NT::DisyTail, {}},
        // 13
        {NT::Conj, {n(NT::Comp), n(NT::ConjTail)}},
        // 14, 15
        {NT::ConjTail, {t(T::KEYWORD_AND), n(NT::Comp), n(NT::ConjTail)}},
        {NT::ConjTail, {}},
        // 16
        {NT::Comp, {n(NT::Suma), n(NT::CompTail)}},
        // 17, 18
        {NT::CompTail, {n(NT::OpComp), n(NT::Suma)}},
        {NT::CompTail, {}},
        // 19-24
        {NT::OpComp, {t(T::EQUAL)}},
        {NT::OpComp, {t(T::NOT_EQUAL)}},
        {NT::OpComp, {t(T::LESS)}},
        {NT::OpComp, {t(T::GREATER)}},
        {NT::OpComp, {t(T::LESS_EQUAL)}},
        {NT::OpComp, {t(T::GREATER_EQUAL)}},
        // 25
        {NT::Suma, {n(NT::Prod), n(NT::SumaTail)}},
        // 26, 27, 28
        {NT::SumaTail, {t(T::PLUS), n(NT::Prod), n(NT::SumaTail)}},
        {NT::SumaTail, {t(T::MINUS), n(NT::Prod), n(NT::SumaTail)}},
        {NT::SumaTail, {}},
        // 29
        {NT::Prod, {n(NT::Unario), n(NT::ProdTail)}},
        // 30, 31, 32
        {NT::ProdTail, {t(T::STAR), n(NT::Unario), n(NT::ProdTail)}},
        {NT::ProdTail, {t(T::SLASH), n(NT::Unario), n(NT::ProdTail)}},
        {NT::ProdTail, {}},
```

```
71          // 33, 34
72          {NT::Unario, {t(T::MINUS), n(NT::Unario)}},
73          {NT::Unario, {n(NT::Primario)}},
74          // 35-39
75          {NT::Primario, {t(T::IDENTIFIER)}},
76          {NT::Primario, {t(T::INTEGER_LITERAL)}},
77          {NT::Primario, {t(T::DECIMAL_LITERAL)}},
78          {NT::Primario, {t(T::STRING_LITERAL)}},
79          {NT::Primario, {t(T::LPAREN), n(NT::Condicion), t(T::RPAREN)}},
80      };
81
82      for (NonTerminal nt : allNonTerminals()) {
83          first_[nt] = {};
84          follow_[nt] = {};
85          nullable_[nt] = false;
86      }
87
88      computeNullable();
89      computeFirst();
90      computeFollow();
91      buildTable();
92  }
```

5. Cálculo de conjuntos FIRST y FOLLOW

Los conjuntos FIRST y FOLLOW fueron calculados algorítmicamente mediante el método de punto fijo implementado en Grammar::computeFirst() y Grammar::computeFollow() (***src/parser_ll1/grammar.cpp***).

No Terminal (A)	Anulable (ε)	Conjunto FIRST(A)	Conjunto FOLLOW(A)
consulta	No	{ SELECT }	{ $ }
proyeccion	No	{ STAR, IDENTIFIER }	{ FROM }
lista_cols	No	{ IDENTIFIER }	{ FROM }
cols_tail	Sí	{ COMMA, ε }	{ FROM }
where_opt	Sí	{ WHERE, ε }	{ SEMICOLON }
condicion	No	{ IDENTIFIER, INTEGER, DECIMAL, STRING, MINUS, LPAREN }	{ SEMICOLON, RPAREN }

disy	No	{ IDENTIFIER, INTEGER, DECIMAL, STRING, MINUS, LPAREN }	{ SEMICOLON, RPAREN }
disy_tail	Sí	{ OR, ε }	{ SEMICOLON, RPAREN }
conj	No	{ IDENTIFIER, INTEGER, DECIMAL, STRING, MINUS, LPAREN }	{ OR, SEMICOLON, RPAREN }
conj_tail	Sí	{ AND, ε }	{ OR, SEMICOLON, RPAREN }
comp	No	{ IDENTIFIER, INTEGER, DECIMAL, STRING, MINUS, LPAREN }	{ AND, OR, SEMICOLON, RPAREN }
comp_tail	Sí	{ EQUAL, NOT_EQUAL, LESS, GREATER, LESS_EQUAL, GREATER_EQUAL, ε }	{ AND, OR, SEMICOLON, RPAREN }
op_comp	No	{ EQUAL, NOT_EQUAL, LESS, GREATER, LESS_EQUAL, GREATER_EQUAL }	{ IDENTIFIER, INTEGER, DECIMAL, STRING, MINUS, LPAREN }
suma	No	{ IDENTIFIER, INTEGER, DECIMAL, STRING, MINUS, LPAREN }	{ op_comp, AND, OR, SEMICOLON, RPAREN }
suma_tail	Sí	{ PLUS, MINUS, ε }	{ op_comp, AND, OR, SEMICOLON, RPAREN }
prod	No	{ IDENTIFIER, INTEGER, DECIMAL, STRING, MINUS, LPAREN }	{ PLUS, MINUS, op_comp, AND, OR, SEMICOLON, RPAREN }

prod_tail	Sí	{ STAR, SLASH, ε }	{ PLUS, MINUS, op_comp, AND, OR, SEMICOLON, RPAREN }
unario	No	{ IDENTIFIER, INTEGER, DECIMAL, STRING, MINUS, LPAREN }	{ STAR, SLASH, PLUS, MINUS, op_comp, AND, OR, SEMICOLON, RPAREN }
primario	No	{ IDENTIFIER, INTEGER, DECIMAL, STRING, LPAREN }	{ STAR, SLASH, PLUS, MINUS, op_comp, AND, OR, SEMICOLON, RPAREN }

```
134  void Grammar::computeFirst() {
135      bool changed = true;
136      while (changed) {
137          changed = false;
138          for (const Production& p : productions_) {
139              std::set<TokenType>& dst = first_[p.lhs];
140              const size_t before = dst.size();
141              for (const Symbol& s : p.rhs) {
142                  if (s.isTerminal()) {
143                      dst.insert(s.terminal);
144                      break;
145                  }
146                  const std::set<TokenType>& src = first_[s.nonterminal];
147                  dst.insert(src.begin(), src.end());
148                  if (!nullable_[s.nonterminal]) break;
149              }
150              if (dst.size() != before) changed = true;
151          }
152      }
153  }
```

```cpp
void Grammar::computeFollow() {
    follow_[startSymbol()].insert(T::END_OF_FILE);  // $ sigue al simbolo inicial
    bool changed = true;
    while (changed) {
        changed = false;
        for (const Production& p : productions_) {
            for (size_t i = 0; i < p.rhs.size(); ++i) {
                if (p.rhs[i].isTerminal()) continue;
                NonTerminal B = p.rhs[i].nonterminal;
                std::set<TokenType>& dst = follow_[B];
                const size_t before = dst.size();

                std::set<TokenType> tail;
                const bool tail_nullable = firstOfSeq(p.rhs, i + 1, tail);
                dst.insert(tail.begin(), tail.end());
                if (tail_nullable) {
                    const std::set<TokenType>& fa = follow_[p.lhs];
                    dst.insert(fa.begin(), fa.end());
                }
                if (dst.size() != before) changed = true;
            }
        }
    }
}
```

6. Tabla de Parseo LL(1) M[A,a]

La tabla de parseo M[A,a] asigna a cada par (No Terminal A,Terminal a) el identificador unívoco de producción a aplicar. Se implementa en Grammar::buildTable() mediante una estructura ***std::map<NonTerminal, std::map<TokenType, int>> table_.***

No Terminal (A)	*Terminal de Entrada (a)*	*Producción Asignada (M[A, a])*
consulta	***SELECT***	***(1) consulta -> SELECT proyeccion FROM IDENTIFIER where_opt ;***
proyeccion	*****	***(2) proyeccion -> ******
	IDENTIFIER	***(3) proyeccion -> lista_cols***
lista_cols	***IDENTIFIER***	***(4) lista_cols -> IDENTIFIER cols_tail***
cols_tail	***,***	***(5) cols_tail -> , IDENTIFIER cols_tail***
	FROM	***(6) cols_tail -> epsilon***

where_opt	*WHERE*	*(7) where_opt -> WHERE condicion*
	;	*(8) where_opt -> epsilon*
condicion	*IDENTIFIER, INTEGER_LITERAL, DECIMAL_LITERAL, STRING_LITERAL, -, (*	*(9) condicion -> disy*
disy	*IDENTIFIER, INTEGER_LITERAL, DECIMAL_LITERAL, STRING_LITERAL, -, (*	*(10) disy -> conj disy_tail*
disy_tail	*OR*	*(11) disy_tail -> OR conj disy_tail*
	;,)	*(12) disy_tail -> epsilon*
conj	*IDENTIFIER, INTEGER_LITERAL, DECIMAL_LITERAL, STRING_LITERAL, -, (*	*(13) conj -> comp conj_tail*
conj_tail	*AND*	*(14) conj_tail -> AND comp conj_tail*
	OR , ; ,)	*(15) conj_tail -> epsilon*
comp	*IDENTIFIER, INTEGER_LITERAL, DECIMAL_LITERAL, STRING_LITERAL, -, (*	*(16) comp -> suma comp_tail*
comp_tail	*=, <>, <, >, <=, >=*	*(17) comp_tail -> op_comp suma*
	AND, OR, ;,)	*(18) comp_tail -> epsilon*
op_comp	*=*	*(19) op_comp -> =*
	<>	*(20) op_comp -> <>*

	<	*(21) op_comp -> <*
	>	*(22) op_comp -> >*
	<=	*(23) op_comp -> <=*
	>=	*(24) op_comp -> >=*
suma	*IDENTIFIER, INTEGER_LITERAL, DECIMAL_LITERAL, STRING_LITERAL, -, (*	*(25) suma -> prod suma_tail*
suma_tail	+	*(26) suma_tail -> + prod suma_tail*
	-	*(27) suma_tail -> - prod suma_tail*
	=, <>, <, >, <=, >=, AND, OR, ;,)	*(28) suma_tail -> epsilon*
prod	*IDENTIFIER, INTEGER_LITERAL, DECIMAL_LITERAL, STRING_LITERAL, -, (*	*(29) prod -> unario prod_tail*
prod_tail	*	*(30) prod_tail -> * unario prod_tail*
	/	*(31) prod_tail -> / unario prod_tail*
	+, -, =, <>, <, >, <=, >=, AND, OR, ;,)	*(32) prod_tail -> epsilon*
unario	-	*(33) unario -> - unario*
	IDENTIFIER, INTEGER_LITERAL, DECIMAL_LITERAL, STRING_LITERAL, (	*(34) unario -> primario*

primario	***IDENTIFIER***	***(35) primario -> IDENTIFIER***
	INTEGER_LITERAL	***(36) primario -> INTEGER_LITERAL***
	DECIMAL_LITERAL	***(37) primario -> DECIMAL_LITERAL***
	STRING_LITERAL	***(38) primario -> STRING_LITERAL***
	(	***(39) primario -> (condicion)***

```
195  void Grammar::buildTable() {
196      for (size_t id = 0; id < productions_.size(); ++id) {
197          const Production& p = productions_[id];
198          std::set<TokenType> firstAlpha;
199          const bool nullAlpha = firstOfSeq(p.rhs, 0, firstAlpha);
200
201          std::set<TokenType> cells = firstAlpha;
202          if (nullAlpha) {
203              const std::set<TokenType>& fa = follow_[p.lhs];
204              cells.insert(fa.begin(), fa.end());
205          }
206
207          for (TokenType a : cells) {
208              auto& row = table_[p.lhs];
209              auto it = row.find(a);
210              if (it != row.end() && it->second != static_cast<int>(id)) {
211                  conflicts_.push_back(std::string("conflicto en M[") + name(p.lhs) + ", " +
212                                       tokenTypeName(a) + "]: producciones " +
213                                       std::to_string(it->second + 1) + " y " +
214                                       std::to_string(id + 1));
215              } else {
216                  row[a] = static_cast<int>(id);
217              }
218          }
219      }
220  }
```

7. Motor Predictivo No Recursivo Basado en Pila

El motor está implementado en la clase **sql::Parser (src/parser_ll1/parser.cpp)**. Opera de forma desacoplada consumiendo sql::LexResult de la Práctica 4. Utiliza una pila explícita **std::vector<Symbol> stack** donde el fondo contiene el centinela **$ (TokenType::END_OF_FILE)** y encima el símbolo inicial **NonTerminal::Consulta.**

```cpp
ParseResult Parser::parse() {
    ParseResult result;

    // Un error lexico impide el analisis sintactico: se propaga como fallo.
    if (!input_.errors.empty()) {
        for (const auto& e : input_.errors) {
            result.errors.push_back({e.message, e.lexeme, e.line, e.column, true});
        }
        return result;
    }

    // Pila con el tope en back(). Fondo: $ (END_OF_FILE); encima el simbolo inicial.
    std::vector<Symbol> stack;
    stack.push_back(Symbol::term(TokenType::END_OF_FILE));
    stack.push_back(Symbol::nonterm(Grammar::startSymbol()));

    size_t pos = 0;
    const auto current = [&]() -> const Token& {
        return input_.tokens[std::min(pos, input_.tokens.size() - 1)];
    };

    // Render del estado actual para la traza.
    const auto renderStack = [&]() {
        std::string out;
        for (const Symbol& s : stack) {  // fondo -> tope (tope a la derecha)
            if (!out.empty()) out += ' ';
            out += Grammar::symbolName(s);
        }
        return out;
    };
```

```cpp
ParseResult Parser::parse() {
    const auto renderInput = [&]() {
        std::string out;
        for (size_t i = pos; i < input_.tokens.size(); ++i) {
            if (!out.empty()) out += ' ';
            const Token& tk = input_.tokens[i];
            out += (tk.type == TokenType::END_OF_FILE) ? "$" : tk.lexeme;
        }
        return out;
    };

    while (!stack.empty()) {
        const Symbol top = stack.back();
        const Token& tok = current();

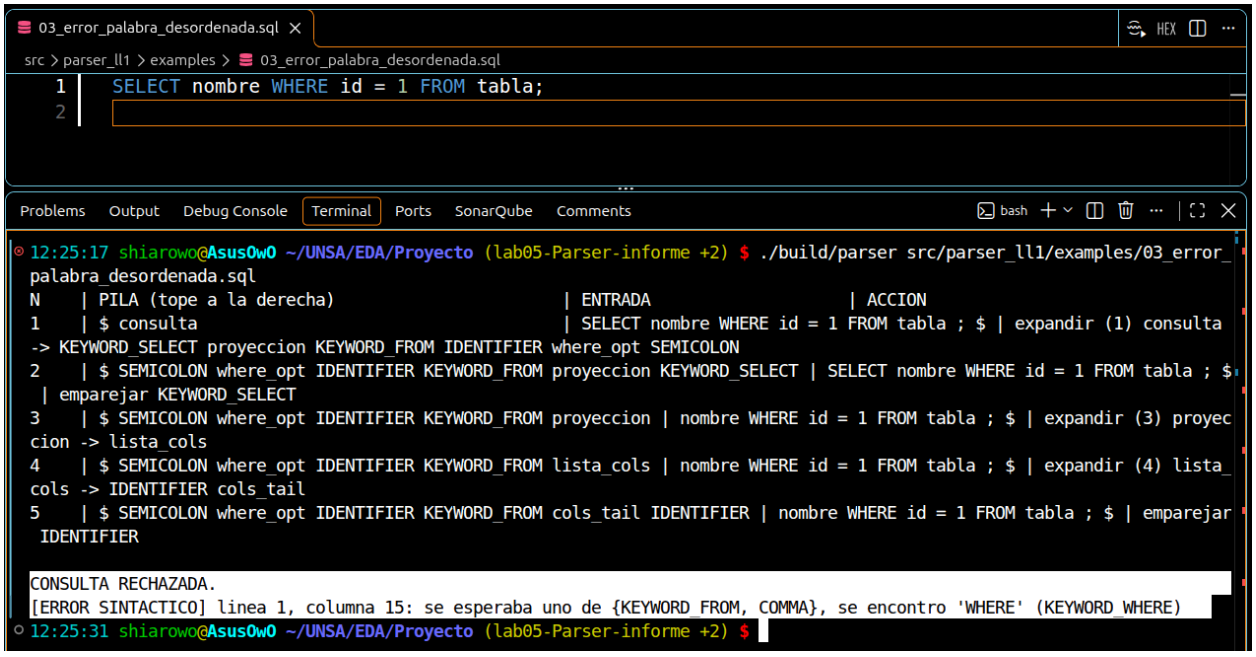
        if (top.isTerminal()) {
            if (top.terminal == tok.type) {
                result.trace.push_back({renderStack(), renderInput(),
                                        "emparejar " + Grammar::symbolName(top)});
                stack.pop_back();
                ++pos;
            } else {
                const std::string expected =
                    top.terminal == TokenType::END_OF_FILE ? "$" : tokenTypeName(top.terminal);
                result.errors.push_back(
                    {"se esperaba " + expected + ", se encontro " + describe(tok),
                     tok.lexeme, tok.line, tok.column, false});
                break;
            }
        } else {
```

```
97          const int id = grammar_.tableEntry(top.nonterminal, tok.type);
98          if (id < 0) {
99              result.errors.push_back(
100                 {"se esperaba uno de {" + joinExpected(grammar_.expected(top.nonterminal)) +
101                     "}, se encontro " + describe(tok),
102                 tok.lexeme, tok.line, tok.column, false});
103             break;
104         }
105         const Production& prod = grammar_.production(id);
106         result.trace.push_back({renderStack(), renderInput(),
107                                 "expandir (" + std::to_string(id + 1) + ") " +
108                                     Grammar::productionString(prod)});
109         stack.pop_back();
110         for (auto it = prod.rhs.rbegin(); it != prod.rhs.rend(); ++it) {
111             stack.push_back(*it);  // empuja el rhs en orden inverso
112         }
113     }
114 }
115
116 result.accepted = result.errors.empty() && stack.empty();
117 return result;
118 }
```

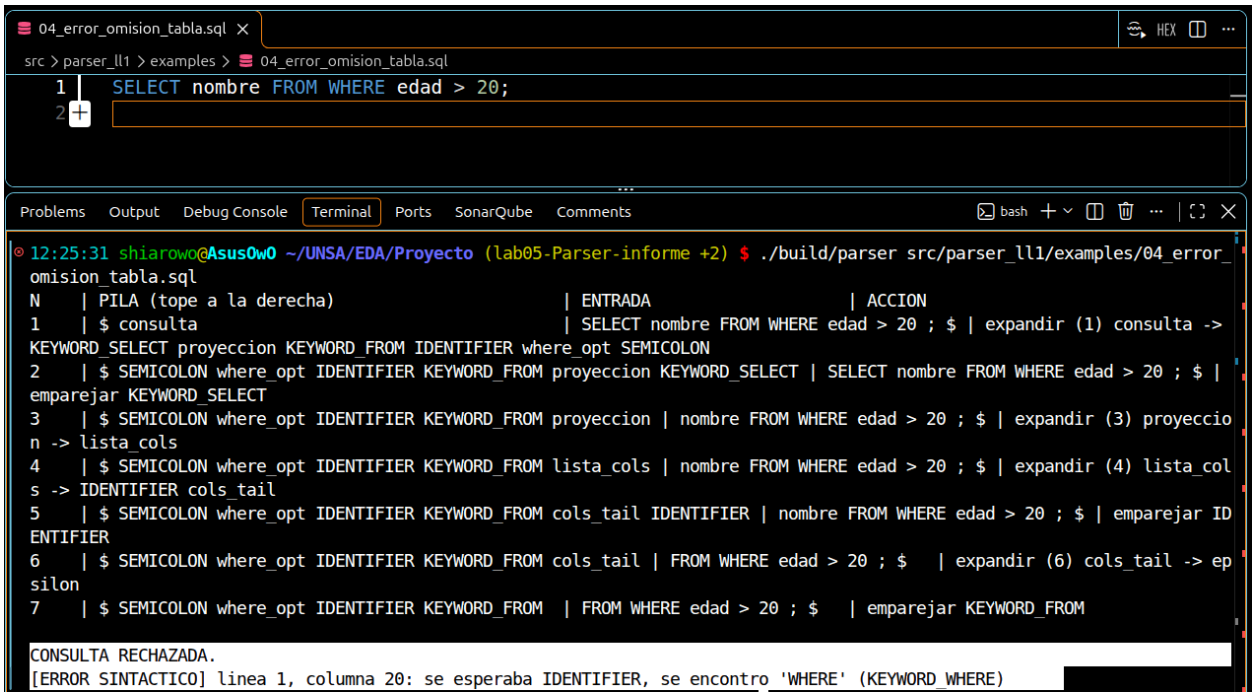
8. Batería de Pruebas y Tratamiento de Errores

Validación de los 4 Tipos de Errores Sintácticos y propagación de errores léxicos

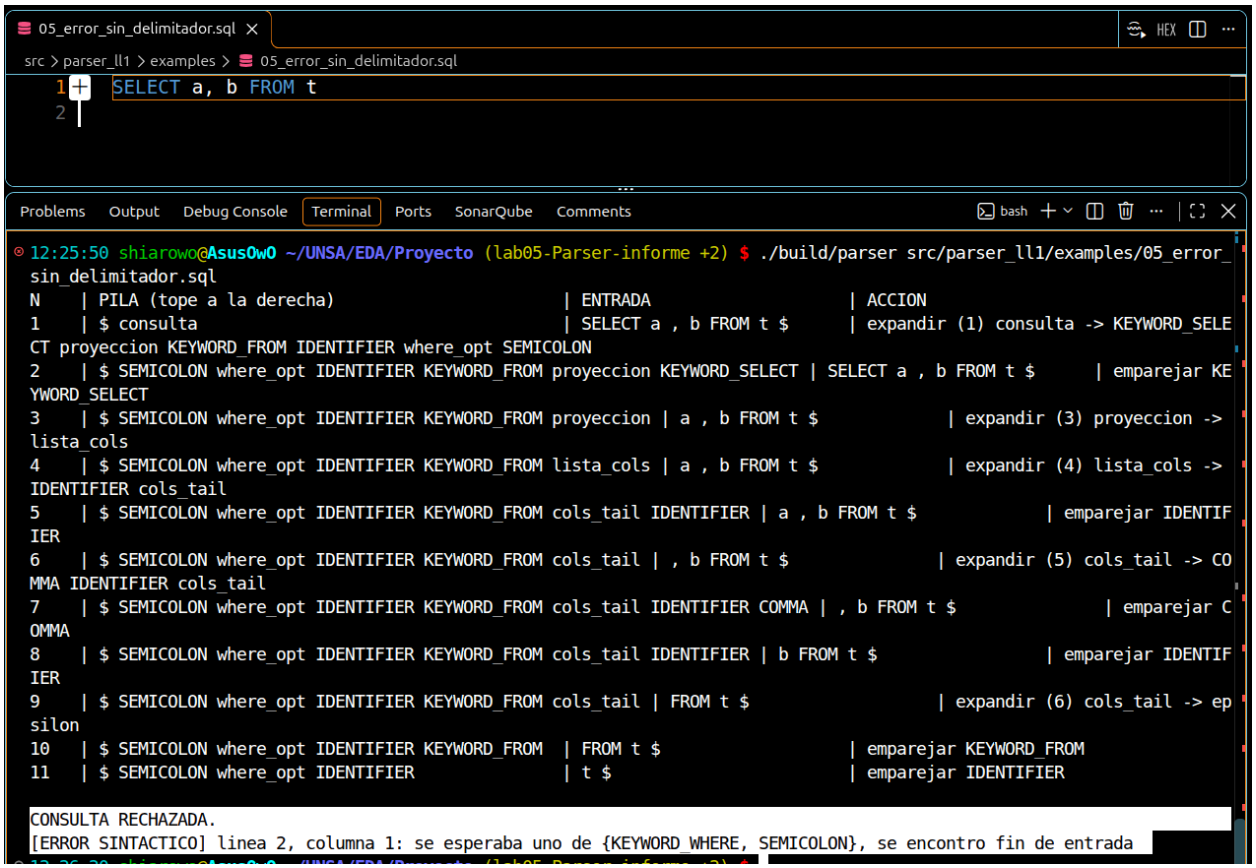
8.1. Caso 1: Palabras clave fuera de orden (WHERE antes de FROM)



8.2. Caso 2: Omisión de tokens obligatorios (nombre de la tabla origen)



8.3. Caso 3: Omisión del delimitador final ;



8.4. Caso 4: Expresiones mal formadas

```
12:32:48 shiarowo@AsusOwO ~/UNSA/EDA/Proyecto (lab05-Parser-informe +2) $ ./build/parser | grep ERROR
SELECT a FROM t WHERE a = ;
[ERROR SINTACTICO] linea 1, columna 27: se esperaba uno de {IDENTIFIER, INTEGER_LITERAL, DECIMAL_LITERAL, STRING_LITERAL, MINUS, L
PAREN}, se encontro ';' (SEMICOLON)
12:33:18 shiarowo@AsusOwO ~/UNSA/EDA/Proyecto (lab05-Parser-informe +2) $ ./build/parser | grep ERROR
SELECT a FROM t WHERE a + > 2;
[ERROR SINTACTICO] linea 1, columna 27: se esperaba uno de {IDENTIFIER, INTEGER_LITERAL, DECIMAL_LITERAL, STRING_LITERAL, MINUS, L
PAREN}, se encontro '>' (GREATER)
12:33:25 shiarowo@AsusOwO ~/UNSA/EDA/Proyecto (lab05-Parser-informe +2) $ ./build/parser | grep ERROR
SELECT a FROM t WHERE (a = 1;
[ERROR SINTACTICO] linea 1, columna 29: se esperaba RPAREN, se encontro ';' (SEMICOLON)
12:33:33 shiarowo@AsusOwO ~/UNSA/EDA/Proyecto (lab05-Parser-informe +2) $
```

8.5. Caso 5: Propagación de errores léxicos

```
06_error_lexico.sql  INFORME_PRACTICA5.md 9+, U
src > parser_ll1 > examples > 06_error_lexico.sql
1  SELECT a # b FROM tabla WHERE x = 12abc;
2

Problems 85  Output  Debug Console  Terminal  Ports  SonarQube  Comments
bash

12:34:37 shiarowo@AsusOwO ~/UNSA/EDA/Proyecto (lab05-Parser-informe +2) $ ./build/parser src/parser_ll1/examples
/06_error_lexico.sql

CONSULTA RECHAZADA.
[ERROR LEXICO] linea 1, columna 10: caracter no admitido
[ERROR LEXICO] linea 1, columna 35: numero seguido de letras (identificador mal formado)
12:34:39 shiarowo@AsusOwO ~/UNSA/EDA/Proyecto (lab05-Parser-informe +2) $
```

9. Ejecución de los test

```cpp
int main() {
    // La gramatica debe ser LL(1): la tabla se construye sin conflictos.
    {
        sql::Grammar g;
        CHECK("gramatica sin conflictos LL(1)", g.conflicts().empty());
        CHECK("tabla tiene entrada M[consulta, SELECT]",
              g.tableEntry(sql::NonTerminal::Consulta, sql::TokenType::KEYWORD_SELECT) >= 0);
        CHECK("celda vacia da -1",
              g.tableEntry(sql::NonTerminal::Consulta, sql::TokenType::KEYWORD_FROM) < 0);
    }

    // Consultas validas (deben ser ACEPTADAS).
    const std::vector<std::string> valid = {
        "SELECT * FROM t;",
        "SELECT nombre, edad FROM estudiantes;",
        "SELECT a FROM t WHERE a = 1;",
        "SELECT * FROM t WHERE a = 1 OR b = 2 AND c > 3;",
        "SELECT x FROM t WHERE (a + 1) > -2.5;",
        "SELECT x FROM t WHERE (a = 1 OR b = 2) AND c <> 'z';",
        "SELECT a, b, c FROM t WHERE a <= b * 2 - 1 AND nombre = 'pepe';",
    };
    for (const auto& source : valid) {
        const auto result = parse(source);
        CHECK(std::string("ACEPTADA: ") + source, result.ok() && !result.trace.empty());
    }
```

```cpp
26  int main() {
52      // Consultas invalidas (deben ser RECHAZADAS, con ubicacion, sin crashear).
53      const std::vector<std::string> invalid = {
54          "SELECT FROM t;",                    // proyeccion vacia
55          "SELECT a, FROM t;",                 // coma sin columna
56          "SELECT a FROM t WHERE a = ;",       // comparacion sin lado derecho
57          "SELECT a FROM t WHERE a + > 2;",    // operador sin operando
58          "SELECT a FROM t WHERE (a = 1;",     // parentesis sin cerrar
59          "SELECT a FROM t",                   // falta ;
60          "SELECT a FROM t; SELECT b FROM t;", // tokens despues del ;
61          "SELECT a FROM 123;",                // tabla no es identificador
62          "FROM t;",                           // no empieza con SELECT
63          "SELECT a WHERE a = 1 FROM t;",      // palabra clave fuera de orden (WHERE antes de FROM)
64          "SELECT *, a FROM t;",               // combinacion invalida de '*' con columnas
65      };
66      for (const auto& source : invalid) {
67          const auto result = parse(source);
68          CHECK(std::string("RECHAZADA: ") + source,
69                !result.ok() && !result.errors.empty() &&
70                result.errors.front().line >= 1 && result.errors.front().column >= 1);
71      }
72
73      // Un error lexico se propaga como fallo del parseo.
74      {
75          const auto result = parse("SELECT a # FROM t;");
76          CHECK("error lexico se propaga",
77                !result.ok() && !result.errors.empty() && result.errors.front().lexical);
78      }
79
80      std::cout << "Resumen: " << passed << " OK, " << failed << " FAIL\n";
81      return failed == 0 ? 0 : 1;
82  }
```

```
12:40:34 shiarowo@AsusOwO ~/UNSA/EDA/Proyecto (lab05-Parser-informe +2) $ cmake --build build
--target test_parser && ./build/test_parser
[ 89%] Built target phase1_core
[100%] Built target test_parser
[OK]   gramatica sin conflictos LL(1)
[OK]   tabla tiene entrada M[consulta, SELECT]
[OK]   celda vacia da -1
[OK]   ACEPTADA: SELECT * FROM t;
[OK]   ACEPTADA: SELECT nombre, edad FROM estudiantes;
[OK]   ACEPTADA: SELECT a FROM t WHERE a = 1;
[OK]   ACEPTADA: SELECT * FROM t WHERE a = 1 OR b = 2 AND c > 3;
[OK]   ACEPTADA: SELECT x FROM t WHERE (a + 1) > -2.5;
[OK]   ACEPTADA: SELECT x FROM t WHERE (a = 1 OR b = 2) AND c <> 'z';
[OK]   ACEPTADA: SELECT a, b, c FROM t WHERE a <= b * 2 - 1 AND nombre = 'pepe';
[OK]   RECHAZADA: SELECT FROM t;
[OK]   RECHAZADA: SELECT a, FROM t;
[OK]   RECHAZADA: SELECT a FROM t WHERE a = ;
[OK]   RECHAZADA: SELECT a FROM t WHERE a + > 2;
[OK]   RECHAZADA: SELECT a FROM t WHERE (a = 1;
[OK]   RECHAZADA: SELECT a FROM t
[OK]   RECHAZADA: SELECT a FROM t; SELECT b FROM t;
[OK]   RECHAZADA: SELECT a FROM 123;
[OK]   RECHAZADA: FROM t;
[OK]   RECHAZADA: SELECT a WHERE a = 1 FROM t;
[OK]   RECHAZADA: SELECT *, a FROM t;
[OK]   error lexico se propaga
Resumen: 22 OK, 0 FAIL
12:40:42 shiarowo@AsusOwO ~/UNSA/EDA/Proyecto (lab05-Parser-informe +2) $
```

10. Conclusiones

- ➔ **Diseño Gramatical Exitoso:** Se diseñó y formalizó una CFG en BNF de 39 producciones libre de recursión izquierda y factorizada. La unificación de condiciones y expresiones a través de la cascada de precedencias resolvió la ambigüedad clásica del paréntesis (, demostrando matemáticamente que los conjuntos de selección son disjuntos
- ➔ **Implementación Robusta del Algoritmo:** El analizador sintáctico predictivo no recursivo implementado en C++17 mediante una pila explícita (std::vector<Symbol>) refleja fielmente el comportamiento de un autómata a pila determinista.
- ➔ **Diagnóstico Preciso y Trazabilidad:** El motor genera diagnósticos completos indicando línea, columna, lexema ofensivo y la lista exacta de tokens esperados en celdas vacías, además de generar trazas detalladas paso a paso para auditoría de la pila.
- ➔ **Integración Modular y Desacoplada:** El analizador consume directamente la estructura *LexResult* de la Práctica 4 sin modificaciones destructivas, completando satisfactoriamente la Fase 2 del servidor relacional SQL.